UNIT 5

INPUT/OUTPUT MANAGEMENT

1. Introduction

In a computer system there are varieties of input and output device with different speed and function. An input/output management module is responsible to manage and control such types of devices. I/O refers to the communication between information processing system, such as a computer, and the outside world possibly a human, or another information processing system. Inputs are the signals or data received by the system, and outputs are the signals or data sent from it. Input output management is also one of the primary responsibilities of an operating system.

2. Principle of I/O Hardware

Different people look input/output hardware in different ways. Electrical engineers look it in term of chips, wires, power supplies and all other physical component that make up the hardware. Programmers look it as the interface presented to the software. The hardware accepts commands, carry out functions and report the result. The role of operating system in computer system is to manage and control I/O operation and I/O devices.

2.1. I/O Devices

These are the devices that communicate with a computer system by sending signals over a cable. An input device is a device by which the computer takes input from the outside world. An input device:

- Accept data from outside world
- Convert data in computer readable form
- Passes the data to the CPU for further action.

An output device is a device by which the computer gives the information to the users.

An output device:

- Receive data from the CPU in computer readable form.
- Convert this information into user readable form.
- Gives the result to the user.

I/O devices can be of two types:

¬ **Block Device:** A device which stores information in fixed size blocks, each one with its own address is termed as block device. Common block size would be range from 512 bytes to 32,768 bytes. The main feature of this device is that it reads or writes each block independently.

Example: Disks are the most common block devices.

¬ **Character Device:** A character device delivers or accepts a stream of characters, without regard to any block structure. It is not addressable and does not have any seek operation.

Example: printer, mouse, etc

2.2. Device Controllers

¬ An electronic device in the form of chip or circuit board that controls functioning of the I/O device is called the device controller or I/O Controller or adopter. The operating system directly deals with the device controller.

¬ It is the hardware that controls the communication between the system and the peripheral drive unit.

¬ Any device connected to the computer is connected by a plug and socket, and the socket is connected to a device controller.

¬ Each device controller has a local buffer and a command register. It communicates with the CPU by interrupts.

¬ A device's controller plays an important role in the operation of that device; it functions as a bridge between the device and the operating system.

¬ I-O devices generally contain two parts: mechanical and electrical part. This electrical part is known as a device controller and can take the form of a chip on personal computers and mechanical part is a device.

¬ It takes care of low-level operations such as error checking, moving disk heads, data transfer, and location of data on the device.

¬ There are many device controllers in a computer system. Example: serial port controller, SCSI bus controller, disk controller

Function of device controllers:

- Stops and starts the activity of the peripheral device.
- Generate error checking code.
- Checks the error in the data received from the interface.
- Abort that command which have errors.
- Retry the command having an error
- Receives the control signals from the interface unit
- Convert the format of the data
- Check the status of the device.

2.3. Memory Mapped I/O

Each controller has a few I/O registers that are used for communicating with the CPU. By writing into these registers, the operating system can command the device to deliver data, accept data, switch itself on or off, or otherwise perform some action. By reading from these registers, the operating system can learn what the device's state is, whether it is prepared to accept a new command, and so on. The system that contains I/O registers as the part of regular memory address space is known as memory mapped I/O. Each control register is assigned a unique memory address to which no memory is assigned. Usually, the assigned addresses are at the top of the address space. In short, the scheme that assigns specific memory location to I/O devices is known as memory mapped I/O. Thus, communication to and from I/O device can be same as reading and writing to memory address that responds to the I/O devices.

2.4. Direct Memory Access (DMA)

DMA Stands for "Direct Memory Access." DMA is a method of transferring data from the computer's RAM to another part of the computer without processing it using the CPU. Most of the data that is input to or output from the computer is processed by the CPU, but some data does not require processing or can be processed by another device. In these situations, DMA can save processing time and is a more efficient way to move data from the computer's memory to other devices. In other words, DMA is a method that allows an I/O device to send or receive data directly to or from the main memory, bypassing the CPU to speed up memory operations. DMA is an essential feature of all modern computers, as it allows devices to transfer data without subjecting the CPU to a heavy overhead. With DMA, the CPU gets freed from this overhead and can-do useful task during data transfer. The process is managed by a chip known as a DMA controller (DMAC). A DMA Controller is a device, which takes over the system bus to directly transfer information from one part of the system to another.

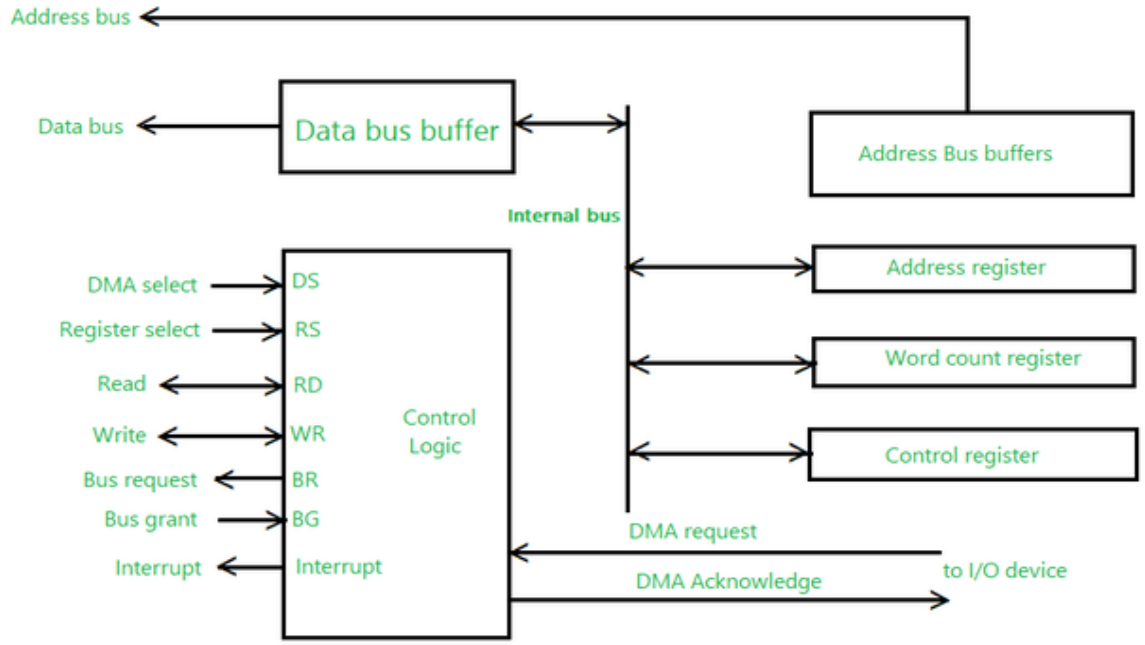


Fig: Working Diagram of DMA Controller

Working of DMA in short

¬ Transfers a block of data directly to or from memory

¬ An interrupt is sent when the task is complete.

¬ The processor is only involved at the beginning and end of the transfer

Advantages of DMA Controller

- Data Memory Access speeds up memory operations and data transfer.
- CPU is not involved while transferring data.
- DMA requires very few clock cycles while transferring data.
- DMA distributes workload very appropriately.
- DMA helps the CPU in decreasing its load.

Disadvantages of DMA Controller

- Direct Memory Access is a costly operation because of additional operations.
- DMA suffers from cache coherence Problems.
- DMA Controller increases the overall cost of the system.
- DMA Controller increases the complexity of the software.

3. Principle Of I/O Software

3.1. Goals of I/O Software

The main goals of I/O software are:

¬ **Device Independence:** It means that it should be possible to write programs that can access any I/O device without having to specify the device in advance.

¬ **Uniform Naming:** The name of a file or a device should simply be a string or an integer and not depend on the device in any way.

¬ **Error Handling:** Errors should be handled as close to the hardware as possible. If the controller discovers a read error, it should try to correct the error itself if it can. If it cannot, then the device driver should handle it.

¬ **Buffering:** Data that come off a device cannot be stored directly in its destination. The data must be put into the buffer before storing in its destination to increase the rate of data transfer.

¬ **Provide a Device Independent Block Size:** The sector size and block size vary from device to device. The I/O software must provide a uniform block size by treating many sectors as a single logical block.

3.2. Program I/O

When the processor is executing a program and encounters an instruction relating to I/O, it executes that instruction by issuing a command to the appropriate I/O module. In programmed I/O, the I/O module will perform the required action and then set the appropriate bits in the I/O status register. The I/O module takes no further action to alert the processor. In particular, it doesn't interrupt the processor. After the I/O instruction is invoked, the processor must periodically check the status of the I/O module until it finds that the operation is complete.

With this technique, the processor is responsible for extracting data from main memory for output and storing data in main memory for input. I/O software is written in such a way that the processor executes instructions that give it direct control of the I/O operation, including sensing device status, sending a read or write command, and transferring the data. Thus, the instruction set includes I/O instructions in the following categories:

Control: Used to activate an external device and tell it what to do. For example, a magnetic tape unit may be instructed to rewind or to move forward one record.

Status: Used to test various status conditions associated with an I/O module and its peripherals.

Transfer: Used to read and/or write data between processor registers and external devices.

3.3. Interrupt Driven I/O

The problem with the programmed I/O is that the processor has to wait a long time for the I/O module of concern to be ready for either reception or transmission of more data. The processor, while waiting, must repeatedly interrogate the status of the I/O module. As a result, the performance level of entire system is degraded.

An alternative approach for this is interrupt driven I/O. in this approach, the processor issues an I/O command to a module and then go on to do some other useful work. The I/O module will then interrupt the processor to request service when it is ready to exchange data with the processor. The processor then executes the data transfer and then resumes its former processing. Interrupt-driven I/O still consumes a lot of time because every data must pass with processor.

The interrupt driven I/O works as follow:

For input, the I/O module receives a READ command from the processor. The I/O module then proceeds to read data in from an associated peripheral. Once the data are in the module's data register, the module signals an interrupt to the processor over a control line. The module then waits until its data are requested by the processor.

3.4. I/O Using DMA

Interrupt-driven I/O is more efficient than simple programmed I/O but it still requires the active intervention of the processor to transfer data between memory and an I/O module, and any data transfer must traverse a path through the processor. Thus both of these forms of I/O suffer from two inherent drawbacks:

¬ The I/O transfer rate is limited by the speed with which the processor can test and service a device.

¬ The processor is tied up in managing an I/O transfer and a number of instructions must be executed for each I/O transfer.

When large volumes of data are to be moved, a more efficient technique is required and this technique is by using DMA. The DMA function can be performed by a separate module on the system bus, or it can be incorporated into an I/O module. In either case, the technique works as follows:

When the processor wishes to read or write a block of data, it issues a command to the DMA module, by sending to the DMA module the following information.

¬ Whether a read or write is requested.

¬ The address of the I/O devices involved.

¬ The starting location in memory to read data from or write data to.

¬ The number of words to be read or written.

The processor then continues with other work. It has delegated the I/O operation to the DMA module. The DMA module transfers the entire block of data, one word at a time, directly to or from memory, without going through the processor.

4. I/O Software Layers

DMA module sends an interrupt signal to the processor. Thus the processor is involved only at the beginning and at the end of the transfer.

The DMA module needs to take control of the bus to transfer data to and from memory.

Thus, a DMA module controls the exchange of data between main memory and an I/O module. The processor sends a request for the transfer of a block of data to the DMA module and is interrupted only after the entire block has been transferred. DMA is far more efficient than interrupt driven or programmed I/O.

CPU

DMA

Do something else

interrupt

DMA

CPU

Next instruction

4.1. Interrupt Handler

It is the bottom layer of I/O software. When an event occurs, the micro-controller generates a hardware interrupt. The interrupt forces the micro-controller's program counter to jump to a specific address in program memory. This special memory address is called the interrupt vector. At this memory location we install a special function known as an interrupt service routine (ISR) which is also known as an interrupt handler. So when a hardware interrupt is generated, program execution jumps to the interrupt handler and executes the code in that handler. When the handler is done, then Read status of DMA module Issues Read/Write block command to I/O module program control returns the micro-controller to the original program it was executing. So a hardware interrupt allows a micro-controller to interrupt an existing program and react to some external hardware event.

4.2. Device Drivers

A driver is a small piece of software that tells the operating system and other software how to communicate with a piece of hardware. A **device driver** is a computer program that operates or controls a particular type of device that is attached to a computer. A device driver provides a software interface to hardware devices, enabling operating systems and other computer programs to access hardware functions without needing to know precise details of the hardware being used. A device driver typically communicates with the device through the computer bus or communications subsystem to which the hardware connects. Device drivers are hardware-dependent and operating system specific.

Each device controller has registers used to give it commands or to read out its status or both.

The number of registers and the nature of the commands vary radically from device to device. Each I/O device attached to a computer needs some device-specific code for controlling it. This

code, called the device driver, is generally written by the device's manufacturer and delivered along with the device on a CD-ROM. Since each operating system needs its own drivers, device manufacturers commonly supply drivers for several popular operating systems. Each device driver normally handles one device type, or one class of closely related devices.

4.3. Device Independent I/O Software

Device independence is the process of making a software application to be able to function on a wide variety of devices i.e. the capability of a program, operating system or programming language to work on varieties of computers or peripherals, despite their electronic variation. The basic function of the device-independent software is to perform the I/O functions that are common to all devices and to provide a uniform interface to the user-level software. It also helps in buffering, error reporting, allocating and releasing dedicated devices and providing a device independent block size.

4.4. User Space I/O Software

It is the upper most layer of I/O system. This layer mainly represents the outermost user interface that occurs between the operating system and end user. A user interface is that portion of an interactive computer system that communicates with the user. Proper design of a user interface can make a significant difference in training time, performance speed, error rates, user satisfaction, and the user's retention of knowledge of operations over time.

5. Disk

5.1. Disk Hardware

A hard disk is part of a computer system that stores and provides relatively quick access to large amounts of data i.e. a disk drive is a randomly addressable and rewritable storage device. A hard disk is really a set of stacked "disks," each of which has data recorded electromagnetically in concentric circles or "tracks" on the disk. A "head" records (writes) or reads the information on the tracks. Two heads, one on each side of a disk, read or write the data as the disk spins. Each read or write operation requires that data be located, which is an operation called a "seek. "**Technical terms related to Disk Drive:**

Platter: Hard drives are normally composed of multiple disks called platters. These platters are stacked on top of each other. It is the platter that actually stores the data.

Spindle/Motor: The platters are attached at the center to a rod or pin called a spindle that is directly attached to the shaft of the motor that controls the speed of rotation.

Head-Actuator Assembly: This assembly consists of an actuator, the arms, the sliders and the read/write heads. The actuator is the device that moves the arms containing read/write heads across the platter surface in order to store and retrieve information. The head arms move between the platters to access and store data. At the end of each arm is a head slider, which consists of a block of material that holds the head. The read/write heads convert the electronic 0s and 1s in the magnetic fields on the disks.

Logic Board: Logic boards consisting of chips, memory and other components control the disk speed and direct the actuator in all its movements. It also performs the process of transferring data from the computer to the magnetic fields on the disk.

Tracks: A track is a concentric ring on the disk where data is stored.

Cylinders: On drives that contain multiple platters, all the tracks on all the platters that are at the same distance from the center are referred to as a cylinder. The data from all the tracks in the cylinder can be read by simply switching between the different heads, which is much faster than physically moving the head between the different tracks on a single disk.

Sectors: Tracks are further broken down into sectors, which are the smallest units of storage on a disk. A larger number of sectors are recorded on the outer tracks, and progressively fewer toward the center. Data can also be read faster from the outer tracks and sectors than the inner ones.

Clusters: Sectors are grouped together into clusters. All the sectors in one cluster are taken as a single unit.

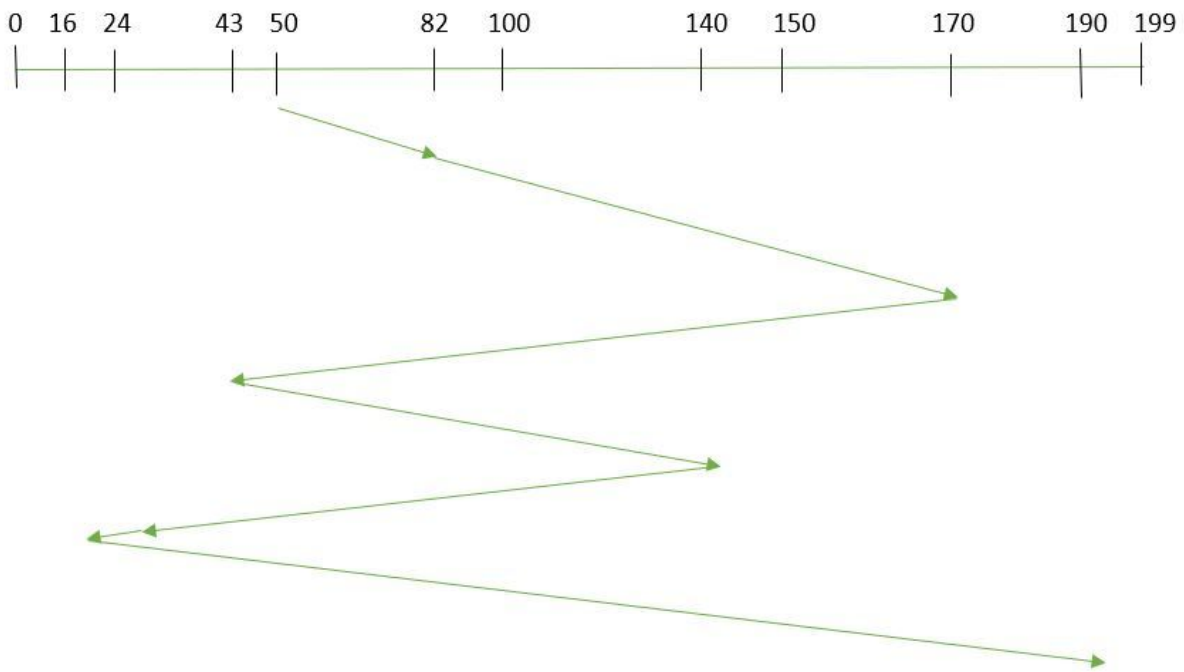
Extents: A set of contiguous clusters storing a single file or part of a file is called an extent. Reducing the number of extents in a file is achieved using a process known as defragmentation, which improves performance.

Bandwidth: The disk bandwidth is the total number of bytes transferred, divided by the total time between the first request for service and the completion of the last transfer.

Disk Management is used to manage the drives (hard disk drives, optical disk drives, flash drives) installed in a computer. Disk Management can be used to partition drives, format drives, assign drive letters, delete partition, and much more.

6. Disk Scheduling

1. FCFS (First Come First Serve)
It is the simplest of all Disk Scheduling Algorithms. In FCFS, the requests are addressed in the order they arrive in the disk queue. Let us understand this with the help of an example.



First Come First Serve
Example:
Suppose the order of request is- (82,170,43,140,24,16,190)
And current position of Read/Write head is: 50

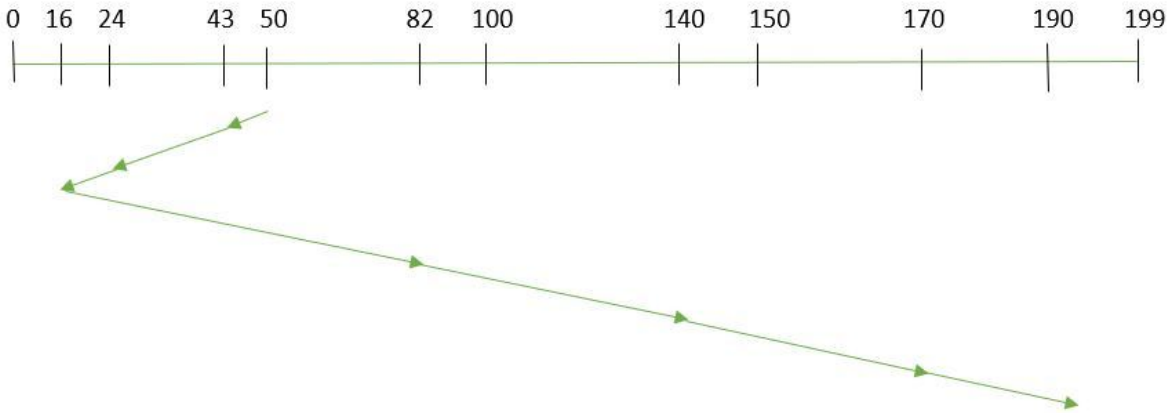
So, total overhead movement (total distance covered by the disk arm) = (82-50) + (170-82) + (170-43) + (140-43) + (140-24) + (24-16) + (190-16) =642
Advantages of FCFS

- Every request gets a fair chance
- No indefinite postponement

Disadvantages of FCFS

- Does not try to optimize seek time
- May not provide the best possible service

2. **SSTF (Shortest Seek Time First)**
Let us understand this with the help of an example.
Example:



Shortest Seek Time First
Suppose the order of request is- (82,170,43,140,24,16,190)
And current position of Read/Write head is: 50
So, total overhead movement (total distance covered by the disk arm) = (50-43)+(43-24)+(24-16)+(82-16)+(140-82)+(170-140)+(190-170) =208

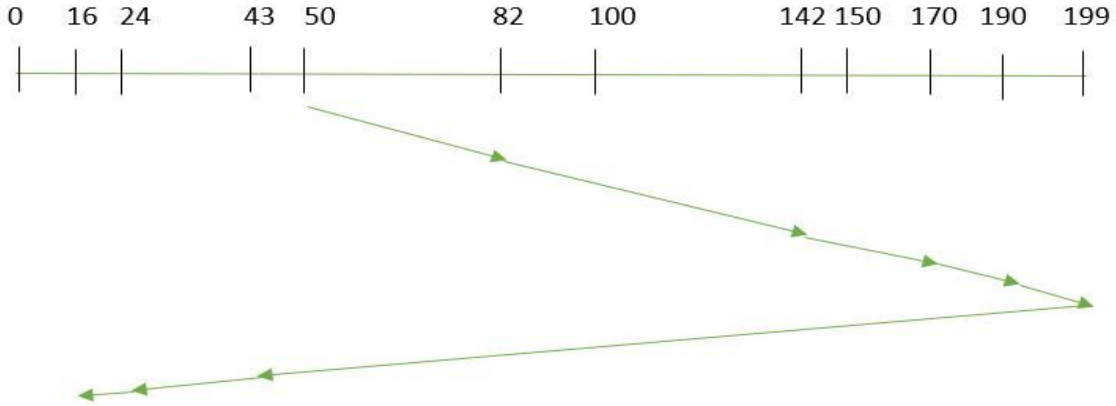
Advantages of Shortest Seek Time First

- The average Response Time decreases
- Throughput increases

Disadvantages of Shortest Seek Time First

- Overhead to calculate seek time in advance
- Can cause Starvation for a request if it has a higher seek time as compared to incoming requests
- The high variance of response time as SSTF Favors only some requests

3. **SCAN**
Example:



Suppose the requests to be addressed are-82,170,43,140,24,16,190. And the Read/Write arm is at 50, and it is also given that the disk arm should move "towards the larger value".
Therefore, the total overhead movement (total distance covered by the disk arm) is calculated as = (199-50) + (199-16) = 332

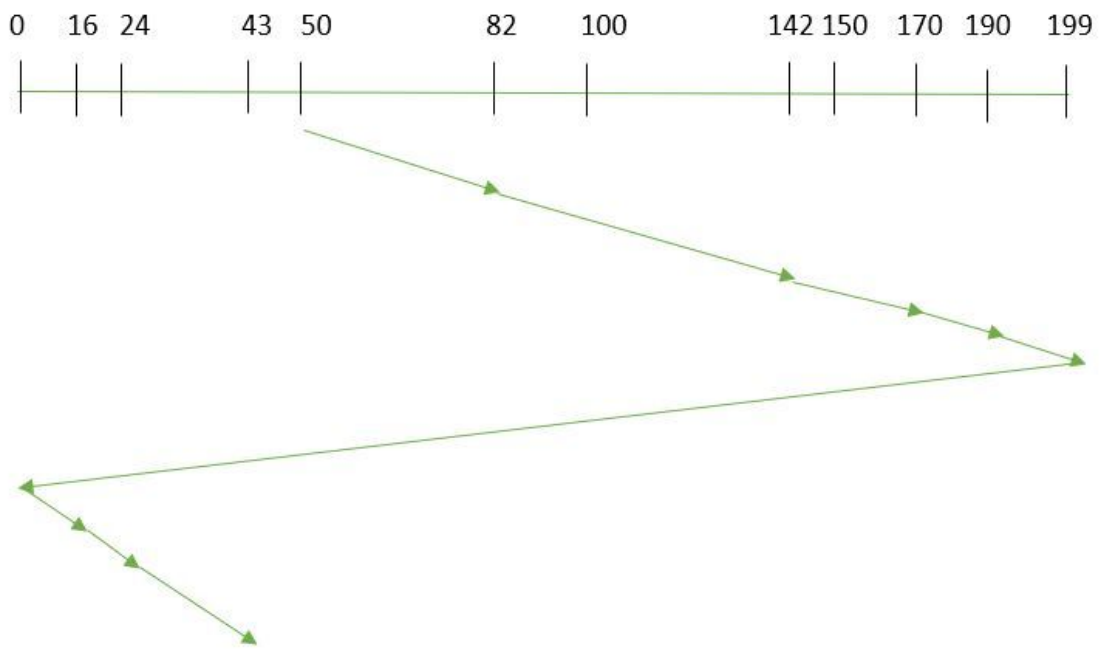
Advantages of SCAN Algorithm

- High throughput
- Low variance of response time
- Average response time

Disadvantages of SCAN Algorithm

- Long waiting time for requests for locations just visited by disk arm

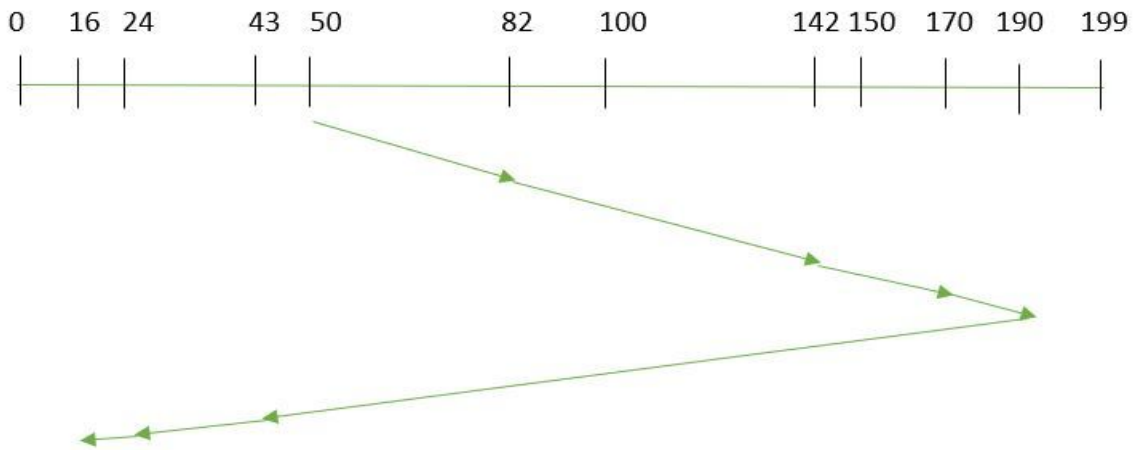
4**. C-SCAN**
Example:



Suppose the requests to be addressed are-82,170,43,140,24,16,190. And the Read/Write arm is at 50, and it is also given that the disk arm should move "towards the larger value".
So, the total overhead movement (total distance covered by the disk arm) is calculated as:
=(199-50) + (199-0) + (43-0) = 391

5. LOOK

Example:



LOOK Algorithm

Suppose the requests to be addressed are-82,170,43,140,24,16,190. And the Read/Write arm is at 50, and it is also given that the disk arm should move "towards the larger value".

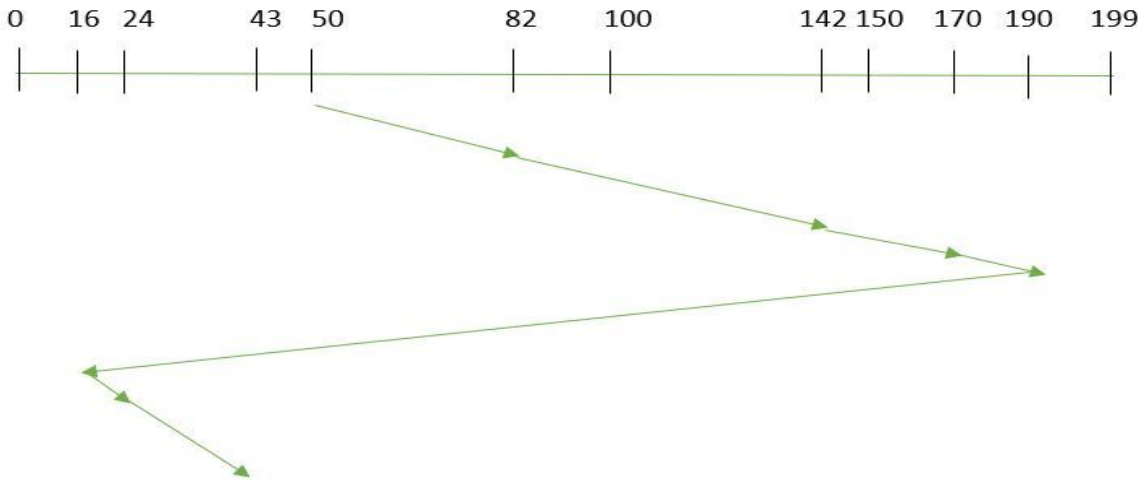
So, the total overhead movement (total distance covered by the disk arm) is calculated as:

= (190-50) + (190-16) = 314

6. C-LOOK

Example:

Suppose the requests to be addressed are-82,170,43,140,24,16,190. And the Read/Write arm is at 50, and it is also given that the disk arm should move "towards the larger value"



C-LOOK

So, the total overhead movement (total distance covered by the disk arm) is calculated as

= (190-50) + (190-16) + (43-16) = 341

RAID

- RAID (Redundant Array of Independent Disks) in a Database Management System (DBMS) is a technology that combines multiple physical disk drives into a single logical unit for data storage. The main purpose of RAID is to improve data reliability, availability, and performance. There are different levels of RAID, each offering a balance of these benefits.

How RAID Works?

Let's understand how RAID (Redundant Array of Independent Disks) works through a relatable example:

Imagine you have a favorite book that you want to keep safe. Instead of giving the entire book to just one friend for safekeeping, you take a smart approach:

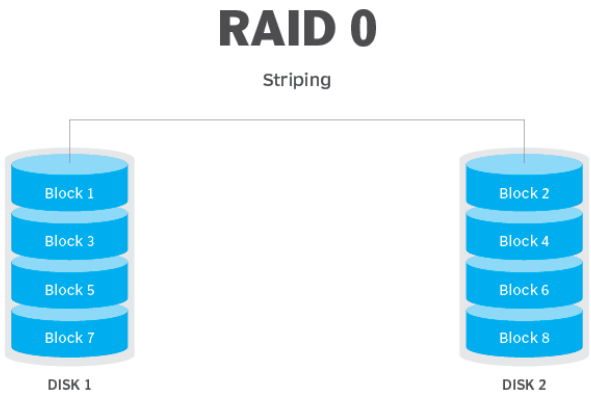
- Splitting the Book: You divide the book into smaller pieces (like chapters or sections) and give each piece to a different friend.
- Making Copies: For extra security, you might also create duplicate pieces and give them to multiple friends.

Different RAID Levels

- RAID 0(Stripping)
- RAID 1(Mirroring)
- RAID-2 (Bit-Level Stripping with Dedicated Parity)
- RAID-3 (Byte-Level Stripping with Dedicated Parity)
- RAID-4 (Block-Level Stripping with Dedicated Parity)
- RAID-5 (Block-Level Stripping with Distributed Parity)
- RAID-6 (Block-Level Stripping with two Parity Bits)

1. RAID-0 (Stripping)

RAID-0 improves system performance by splitting data into smaller "blocks" and spreading them across multiple disks. This process is called "striping." It enhances data access speed by enabling parallel read/write operations but provides no redundancy or fault tolerance.



Advantages

It is easy to implement.

It utilizes the storage capacity in a better way.

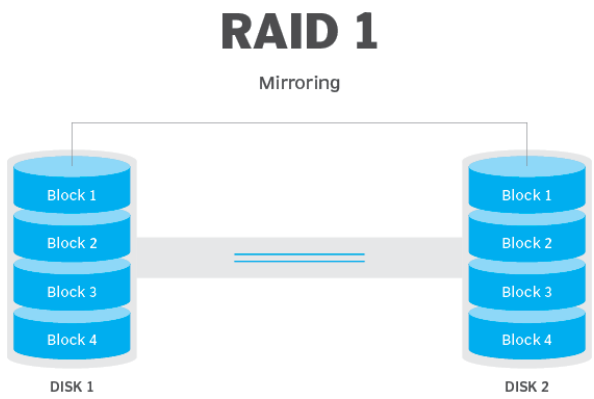
Disadvantages

A single drive loss can result in the complete failure of the system.

It's not a good choice for a critical system.

2. RAID-1 (Mirroring)

RAID-1 enhances reliability by creating an identical copy (mirror) of each data block on separate disks. This ensures that even if one disk fails, the data remains accessible from its duplicate. While this configuration is highly reliable, it requires significant storage overhead.



Advantages

It covers complete redundancy.

It can increase data security and speed.

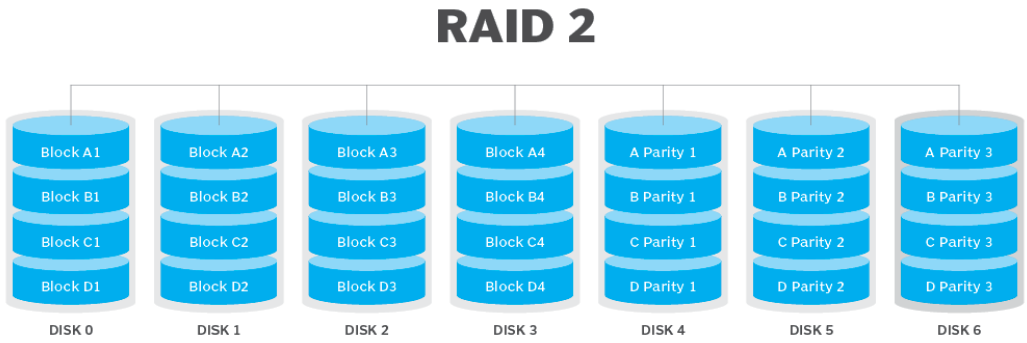
Disadvantages

It is highly expensive.

Storage capacity is less.

3. RAID-2 (Bit-Level Stripping with Dedicated Parity)

RAID-2 is a specialized RAID level that uses bit-level striping combined with error correction using hamming code. In this configuration, data is distributed at the bit level across multiple drives, and a dedicated parity drive is used for error detection and correction.



Advantages

In case of Error Correction, it uses hamming code.

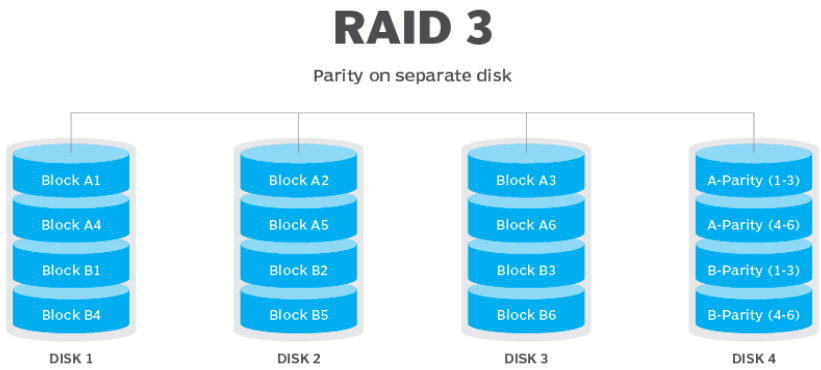
It Uses one designated drive to store parity.

Disadvantages

It has a complex structure and high cost due to extra drive.

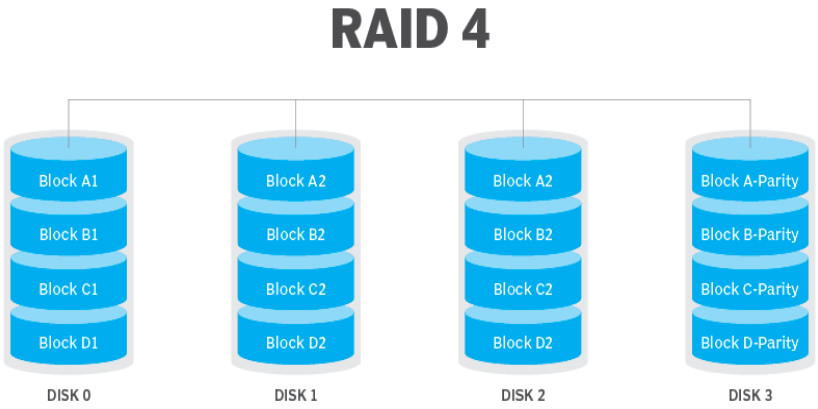
It requires an extra drive for error detection.

4. RAID-3 (Byte-Level Stripping with Dedicated Parity)
RAID-3 enhances fault tolerance by employing byte-level striping across multiple drives and storing parity information on a dedicated parity drive. The dedicated parity drive allows for the reconstruction of lost data if a single drive fails.



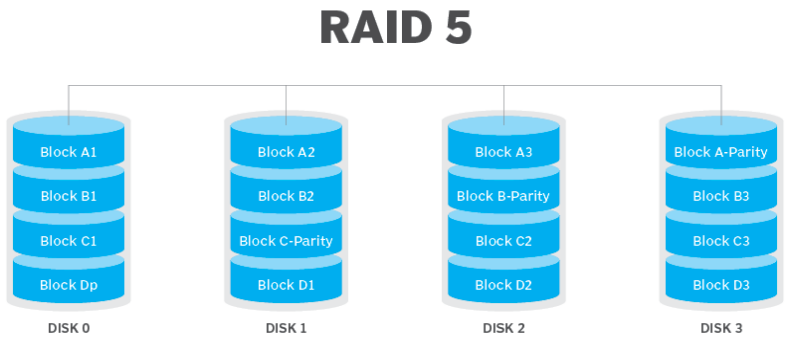
Advantages
Data can be transferred in bulk.
Data can be accessed in parallel.
Disadvantages
It requires an additional drive for parity.
In the case of small-size files, it performs slowly.

5. RAID-4 (Block-Level Stripping with Dedicated Parity)
RAID-4 introduces block-level striping across multiple disks, combined with a dedicated parity disk to provide fault tolerance. Data is written in blocks, and a separate disk stores parity information calculated using the XOR function.



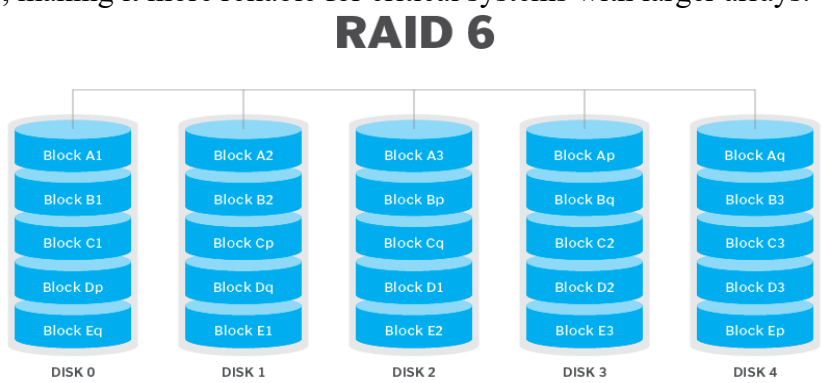
- **Advantages**:
  - Fault tolerance; can withstand the failure of one disk.
  - Good balance of speed and redundancy.
- **Disadvantages**:
  - Rebuilding a failed disk takes time.
  - Write performance is slower due to parity calculations.

6. RAID-5 (Block-Level Stripping with Distributed Parity)
RAID-5 builds on RAID-4 by distributing parity information across all disks instead of storing it on a dedicated parity drive. This distributed parity significantly improves write performance, especially for random write operations, while maintaining fault tolerance for single disk failures.



Advantages
Data can be reconstructed using parity bits.
It makes the performance better.
Disadvantages
Its technology is complex and extra space is required.
If both discs get damaged, data will be lost forever.

7. RAID-6 (Block-Level Stripping with two Parity Bits)
RAID-6 is an advanced version of RAID-5 that provides enhanced fault tolerance by introducing double distributed parity. This allows RAID-6 to recover from the failure of up to two disks simultaneously, making it more reliable for critical systems with larger arrays.



Advantages
Very high data Accessibility.
Fast read data transactions.
Disadvantages
Due to double parity, it has slow written data transactions.
Extra space is required.